



UNIVERSIDAD DE EXTREMADURA

FACULTAD DE CIENCIAS

Grado en Física

MEMORIAS DE PRÁCTICAS EXTERNAS

SARAH GONZÁLEZ ABADITO
JULIO, 2025

ÍNDICE

1. Introducción	2
1.1 Resumen	2
1.2 Descripción de la Empresa	2
1.3 Objetivos	2
2. Estado del arte	3
3. Formación Inicial	3
3.1 Redes y MQTT	3
3.2 FIWARE y Postman	5
3.3 Node-RED	7
3.4 Análisis de Datos con KNIME	8
3.5 Arduino	9
3.6 Python	10
4. Proyectos Realizados	11
4.1 Proyecto con Postman	11
4.2 Alarma con Node-RED	12
4.3 Proyecto con KNIME	13
4.4 Evento en Olivenza	17
4.5 Proyecto Final con Arduino	18
4.6 Python	23
5. Conclusión	25
6. Bibliografía	26

1. Introducción

1.1 Resumen

En esta memoria se documentan todos los conocimientos adquiridos y actividades realizadas durante el periodo de prácticas externas en el Centro FIWARE Space.

Se incluye una breve descripción de la empresa, en la que se especifican sus áreas de trabajo, objetivos y tecnologías que utilizan. A continuación, se exponen los objetivos para las prácticas, así como los conocimientos adquiridos y proyectos realizados a partir de ellos. Todo ello se acompaña por imágenes, tablas y explicaciones detalladas. Por último, el documento finaliza con una conclusión y reflexión personal sobre la experiencia.

1.2 Descripción de la Empresa

FIWARE Space es un centro de emprendimiento tecnológico, perteneciente a un proyecto global impulsado por la Diputación de Badajoz llamado “Badajoz es Más”. Su objetivo principal es mejorar la eficiencia y las tecnologías de áreas rurales, con el fin de modernizar los servicios públicos y la administración de los municipios de la provincia.

El Centro se encarga de crear soluciones para diversas áreas como ciudades inteligentes, industria, agricultura y energías renovables, utilizando la plataforma de código abierto Fiware. Para ello, emplea herramientas como la sensorización y el análisis de datos.

Además, actúa como consultoría tecnológica, ofreciendo programas de mentorización para empresas y todo tipo de entidades. Se encargan de impartir los conocimientos necesarios para implementar tecnologías adaptadas a cada caso, realizando un acompañamiento continuo durante todo el proceso.

Asimismo, ofrecen cursos todas las semanas y colaboran con personas desempleadas y jóvenes en sus comienzos en el ámbito laboral, incluyendo estudiantes en prácticas.

1.3 Objetivos

El principal motivo por el que decidí realizar estas prácticas fue para tener una primera toma de contacto con el entorno laboral, especialmente dentro del ámbito tecnológico. Mi objetivo fue poner en práctica los conocimientos adquiridos en la universidad y, al mismo tiempo, adquirir nuevas habilidades que no siempre se enseñan en el aula.

Busqué aprender a desenvolverme en un entorno profesional real, usar distintas herramientas y metodologías tecnológicas actuales y entender mejor cómo se gestionan los proyectos en una organización. En definitiva, complementar mi formación académica con una experiencia práctica que me permitiera crecer tanto a nivel técnico como personal.

2. Estado del Arte

En la actualidad, la mayor parte de las soluciones tecnológicas que utilizamos a diario se basan en el tratamiento de datos, la automatización de procesos o la conexión entre dispositivos. En este contexto, algunas herramientas como FIWARE, Node-RED o Python se han convertido en opciones muy utilizadas tanto a nivel académico como profesional.

Por un lado, tenemos FIWARE, una plataforma de código abierto pensada para construir soluciones inteligentes, sobre todo en áreas como ciudades inteligentes, agricultura o industria. Por otro lado, Node-RED permite crear flujos de datos visuales de forma sencilla, facilitando el desarrollo de pequeños sistemas sin necesidad de escribir grandes cantidades de código. Y Python es uno de los lenguajes más populares hoy en día por su versatilidad, fácil uso y cantidad de librerías disponibles.

Durante mis prácticas, he podido trabajar con todas estas tecnologías, lo que me ha permitido entender mejor su utilidad, así como el papel que juegan en la creación de proyectos reales.

3. Formación Inicial

3.1 Redes y MQTT

En nuestro primer día, nuestro tutor Jorge Osuna nos enseñó conceptos básicos sobre redes y conexiones.

Nos comentó que una red está formada por un conjunto de dispositivos que se conectan entre sí para compartir información, trabajar juntos o acceder a servicios. Además, todos los dispositivos de una red tienen una Dirección IP (Internet Protocol), que es un número único identificativo, similar al DNI. Estas pueden ser de dos tipos:

- **Dirección IP fija:** no cambia, puede configurarse manualmente y es muy útil para servidores.
- **Dirección IP dinámica:** cambia cada vez que el dispositivo se conecta a la red. La asigna el router automáticamente.

Además, también existen varios tipos de redes que podemos clasificar según su alcance y cantidad de dispositivos conectados:

- **Redes de área local (LAN):** Son redes que solo funcionan dentro de un espacio cerrado, como una casa, empresa, o laboratorio.
- **Redes de área amplia (WAN):** Aquí pertenecen las redes de larga distancia, como internet.
- **Redes personales (PAN):** Son muy pequeñas, como las conexiones Bluetooth.

Igual que cada dispositivo dentro de una red necesita una dirección IP para poder identificarse, la red completa también se puede identificar con una IP de red.

Otro concepto importante es la máscara de subred, que sirve para dividir una red en subredes más pequeñas. Una máscara común es 255.255.255.0, donde los primeros 3 grupos de la IP definen la red, y el último grupo define los dispositivos dentro de ella.

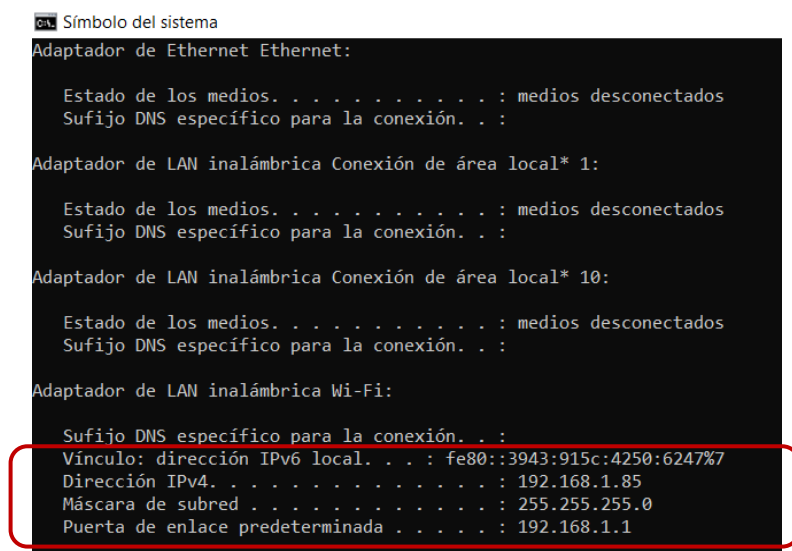
Por ejemplo, la red 192.168.1.x, los dispositivos van del .1 al .254. Esta IP (192.168.1.0) es una dirección de red local que puede contener dispositivos como 192.168.1.1 (router) 192.168.1.10 (PC) o 192.168.1.20 (sensor).

Además, existen los puertos, que permiten la comunicación de varios servicios a través de la misma IP. Cabe destacar que un ordenador puede usar la misma IP para múltiples tareas si cada una usa un puerto distinto. Algunos puertos típicos son:

- **3306**: puerto de base de datos SQL.
- **1880**: puerto típico del Node-RED, con el que trabajaremos más adelante.
- **1883**: puerto estándar de MQTT, del que hablaremos a continuación.
- **1026**: usado por FIWARE, plataforma de la que hablaremos en la siguiente sección.

También tenemos la dirección *Localhost* o *IP 127.0.0.1*, que siempre apunta a nuestro propio ordenador y se usa para hacer pruebas locales sin conectarse a una red externa.

Podemos ver la IP de nuestro PC abriendo el CMD y escribiendo *ipconfig*. Al pulsar, vamos a ver una serie de especificaciones, entre las que se encuentra la dirección IP del dispositivo, como se muestra en la **Imagen 3.1.1**



```
Símbolo del sistema
Adaptador de Ethernet Ethernet:

    Estado de los medios. . . . . : medios desconectados
    Sufijo DNS específico para la conexión. . . :

Adaptador de LAN inalámbrica Conexión de área local* 1:

    Estado de los medios. . . . . : medios desconectados
    Sufijo DNS específico para la conexión. . . :

Adaptador de LAN inalámbrica Conexión de área local* 10:

    Estado de los medios. . . . . : medios desconectados
    Sufijo DNS específico para la conexión. . . :

Adaptador de LAN inalámbrica Wi-Fi:

    Sufijo DNS específico para la conexión. . . :
    Vínculo: dirección IPv6 local. . . : fe80::3943:915c:4250:6247%7
    Dirección IPv4. . . . . : 192.168.1.85
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . . : 192.168.1.1
```

Imagen 3.1.1 – Dirección IP del PC.

Más adelante, en el contexto del IoT (Internet of things), Jorge nos explicó el funcionamiento del protocolo MQTT. Este es un protocolo de mensajería utilizado para la comunicación de un equipo a otro. Los sensores inteligentes, los dispositivos portátiles y otros dispositivos de IoT generalmente tienen que transmitir y recibir datos a través de una red con recursos restringidos y un ancho de banda limitado. Por esta razón, MQTT es ideal para la transmisión de datos con dispositivos, ya que es eficiente y fácil de implementar. Para entender mejor el funcionamiento de este protocolo es necesario definir dos conceptos importantes:

- **Cliente MQTT**: es cualquier dispositivo que utiliza una biblioteca compatible con el protocolo MQTT para establecer comunicación a través de una red. Dependiendo de su función, el cliente puede comportarse como publicador (cuando envía mensajes) o como suscriptor (cuando los recibe).
- **Agente MQTT**: también conocido como broker, es el sistema central encargado de gestionar la comunicación entre los distintos clientes.

Además, utilizamos Mosquitto, un software de código abierto que implementa un agente MQTT o broker. Su función principal consiste en recibir mensajes publicados por clientes MQTT, identificar qué otros clientes están suscritos al tema correspondiente y reenviar dichos mensajes exclusivamente a ellos.

3.2 FIWARE y Postman

Unos días después, recibimos formación básica de FIWARE con ayuda de nuestro tutor Miguel.

FIWARE es una plataforma de código abierto para el desarrollo de aplicaciones inteligentes impulsada por la Unión Europea, siendo nuestro centro uno de los 31 existentes colaboradores en todo el mundo.

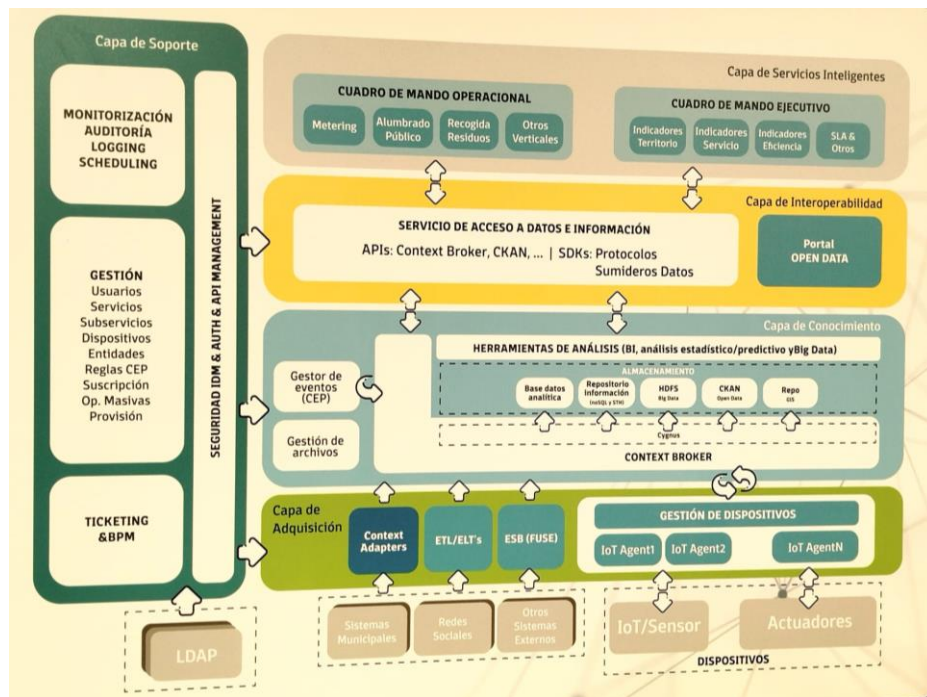


Imagen 3.2.1 – Arquitectura FIWARE.

La **Imagen 3.2.1** representa la arquitectura de una plataforma inteligente basada en FIWARE. Tenemos una primera capa de dispositivos físicos, normalmente sensores/actuadores, que recogen la información del mundo físico. Estos se comunican a través de Iot Agents que entienden distintos protocolos como MQTT o HTML.

Pasamos a la segunda capa, de adquisición de datos, donde los Iot Agents traducen lo que dicen los dispositivos a un lenguaje que entienda el Context Broker. También tenemos los ETL/ELT, Context Adapters, y ESB (FUSE) que permiten adaptar o transformar datos antes de enviarlos al sistema. Desde aquí, los datos llegan al Context Bróker que es el corazón de la plataforma. Es el encargado de gestionar el contexto: almacena el estado de sensores, permite crear, borrar, actualizar y leer entidades, así como notificar cambios. Es como una base de datos dinámica que cambia con el estado de los sensores.

Nosotros trabajaremos con *Orion Context Broker*, utilizando la aplicación *Postman*, que envía peticiones HTTP como:

- **GET** → Consultar datos.
- **POST** → Crear datos nuevos (por ejemplo, entidades o sensores).
- **PUT** → Modificar datos existentes.
- **DELETE** → Borrar entidades.

Orion Context Broker utiliza el puerto 1026 por defecto para recibir y enviar datos en formato JSON. Este formato se caracteriza por tener elementos par clave – valor, por ejemplo:

```
{
  "temperatura": 22.5
}
```

Donde la clave es una cadena entre comillas y el valor puede ser un número, texto, booleano, o una lista.

Para crear una entidad en FIWARE usando Postman, debemos hacer una solicitud *POST* al context Broker. Imaginemos que tenemos sensorizada una piscina climatizada, podemos crear una entidad de la piscina, asignándole un ID y un tipo, como se muestra en la **Imagen 3.2.2**.

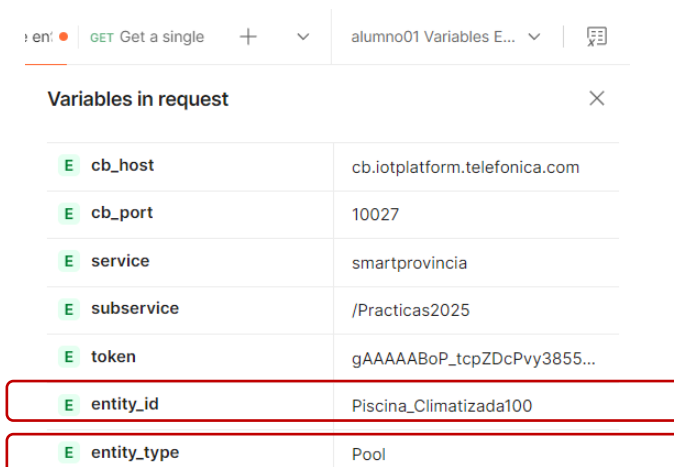


Imagen 3.2.2 – ID y tipo de la entidad.

A continuación, debemos crearle atributos, que son las variables medidas con el sensor que cambian a lo largo del tiempo. Para ello, nos vamos a *POST Create entity* y escribimos en formato JSON los atributos, con un valor inicial y el tipo (número, string, booleano). Una vez hecho esto, le damos a enviar (*send*), si todo ha ido bien, debería aparecer un código 201, tal como aparece en **Imagen 3.2.3**.

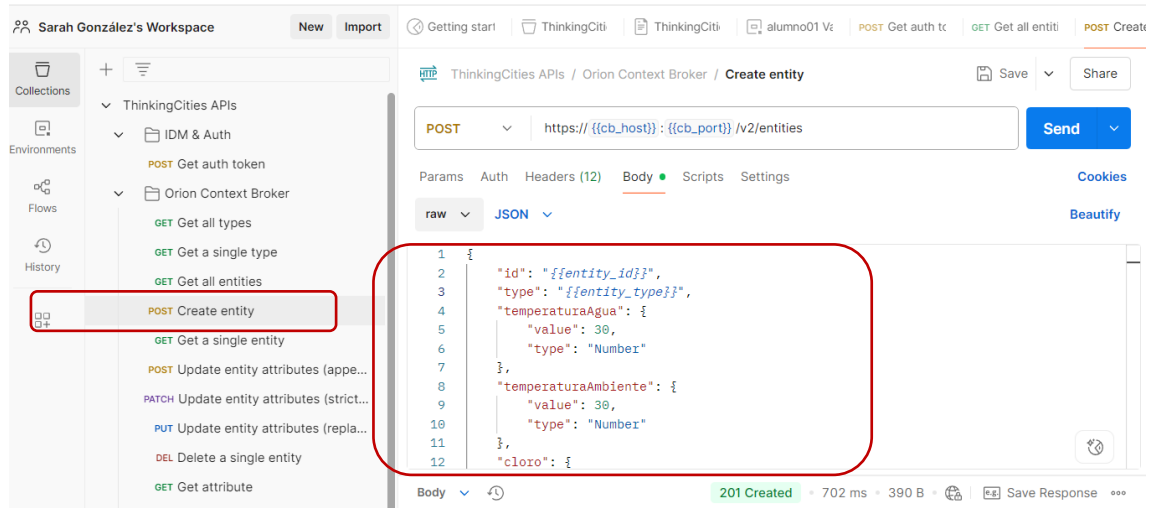


Imagen 3.2.3 – Atributos de la entidad.

Podemos ver todas las entidades que creamos en *GET all entities*. Además, existe la opción de crear una suscripción, ello permite que el context bróker nos notifique cuando haya alguna actualización en los atributos de las entidades creadas.

Veremos con más detalle todo esto en la sección 4.1.

3.3 Node-RED

Durante uno de los días de nuestras prácticas, Jorge nos explicó cómo utilizar Node-RED, una herramienta de programación basada en flujos, muy útil para conectar dispositivos dentro del Internet de las Cosas. Está construida sobre la plataforma Node.js, de forma que todos los flujos y operaciones se ejecutan utilizando JavaScript.

Para trabajar en Node-RED, se utiliza un entorno gráfico accesible desde un navegador web, donde podemos arrastrar, soltar y conectar nodos para formar flujos de datos. Cada nodo tiene una función específica (entrada, salida, procesamiento, conexión, etc.), y al conectarlos entre sí se define el recorrido de la información dentro del sistema.

Trabajamos con esta herramienta posteriormente, en la sección 4.2.

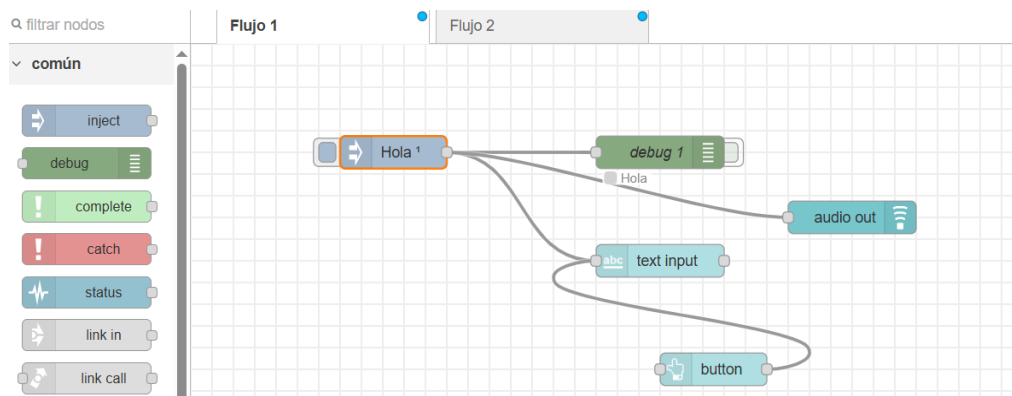


Imagen 3.3.1 – Flujo en Node-RED.

3.4 Análisis de Datos con KNIME

En los últimos años, la importancia del análisis de datos ha crecido bastante, debido a su enorme capacidad para transformar grandes volúmenes de información en conocimiento útil y accesible. Nos permite extraer y analizar datos relevantes, identificar patrones y optimizar procesos, resultando fundamental para una toma de decisiones más eficiente.

Además, el análisis de datos abarca un amplio conjunto de técnicas y metodologías, entre las que se destacan la estadística descriptiva, la inferencia estadística, el aprendizaje automático y la minería de datos.

En mi caso, comencé a introducirme en este campo utilizando KNIME, una plataforma de análisis de datos de código abierto que permite trabajar con grandes cantidades de datos sin necesidad de programar.

KNIME no se basa en la escritura de scripts, sino que funciona mediante un entorno gráfico de flujos de trabajo (workflows), muy parecido en estilo a Node-RED. Cada operación se representa mediante un pequeño bloque llamado nodo, encargado de ejecutar una tarea específica. Al conectar estos nodos entre sí, se define una secuencia lógica para el procesamiento de los datos.

Un nodo toma un conjunto de datos como entrada, lo procesa y lo pone a disposición en su puerto de salida. El tipo de operación realizada por un nodo es muy amplio y puede ir desde tareas básicas como la lectura o escritura de archivos, hasta técnicas más complejas como el modelado de datos mediante redes neuronales, o la visualización gráfica de resultados mediante diagramas de dispersión, histogramas y otros tipos de gráficas.

Cada nodo en KNIME puede tener 4 estados, representado por diferentes colores:

- | | |
|----------------------------------|-------------|
| - Inactivo y no configurado | → Rojo |
| - Configurado, pero no ejecutado | → Amarillo |
| - Ejecutado exitosamente | → Verde |
| - Ejecutado con errores | → Cruz roja |

A continuación, se muestran en la **Imagen 3.4.1** cuatro ejemplos del mismo nodo (un nodo lector de archivos) en cada uno de los cuatro estados:



Imagen 3.4.1 – Nodo en cada estado.

3.5 Arduino

Arduino es una plataforma de hardware y software de código abierto diseñada para crear proyectos electrónicos de una forma sencilla y accesible.

El hardware está formado por una placa con un microcontrolador que puede leer entradas (como sensores o botones) y controlar salidas (como LEDs, motores o pantallas). El software, por su parte, es un programa llamado Arduino IDE, donde se escribe el código y se envía a la placa para que lo ejecute. Además, al tratarse de un hardware libre, permite que la gente pueda estudiarlo para entender su funcionamiento, modificarlo, reutilizarlo, mejorarlo y compartir dichos cambios.

Su origen se remonta al año 2005 en el Instituto de Diseño Interactivo de Ivrea (Italia), centro académico donde los estudiantes se dedicaban a experimentar con diferentes dispositivos (muchos de ellos basados en microcontroladores) para conseguir generar espacios únicos. Arduino apareció por la necesidad de contar con un dispositivo para utilizar en las aulas que fuera de bajo coste, que funcionase bajo cualquier sistema operativo y que contase con documentación adaptada a gente que quisiera empezar de cero.

El voltaje de funcionamiento de la placa Arduino (incluyendo el microcontrolador y el resto de las componentes) es de 5 V. Podemos obtener esta alimentación eléctrica de varias maneras:

- Conectando la placa Arduino a una fuente externa, tal como un adaptador AC/DC o una pila.
- Conectando la placa Arduino a nuestro PC mediante un cable USB.

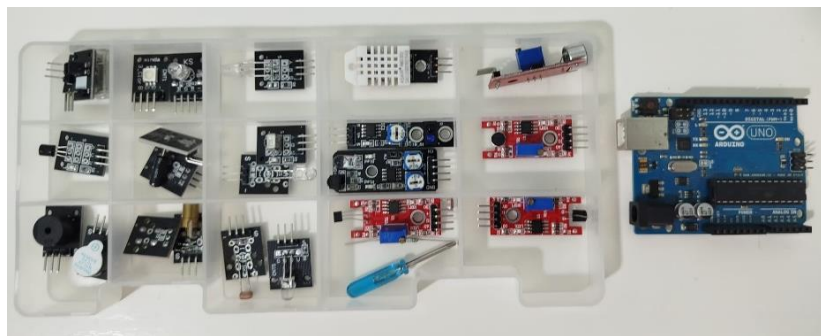


Imagen 3.5.1- Placa Arduino y Kit de Sensores.

Un sketch en Arduino se divide en tres secciones principales:

- **Declaración de variables globales:** Se colocan al inicio del programa, fuera de cualquier función. Estas variables estarán disponibles en todo el código.
- **Void setup():** Esta función se ejecuta una sola vez al encender o reiniciar la placa. Se utiliza para inicializar elementos (por ejemplo, definir pines como entrada o salida, o iniciar la comunicación serie).
- **Void loop():** Es la función que se repite continuamente mientras la placa esté encendida. Aquí se escribe el código principal que deseamos que se ejecute de forma cíclica.

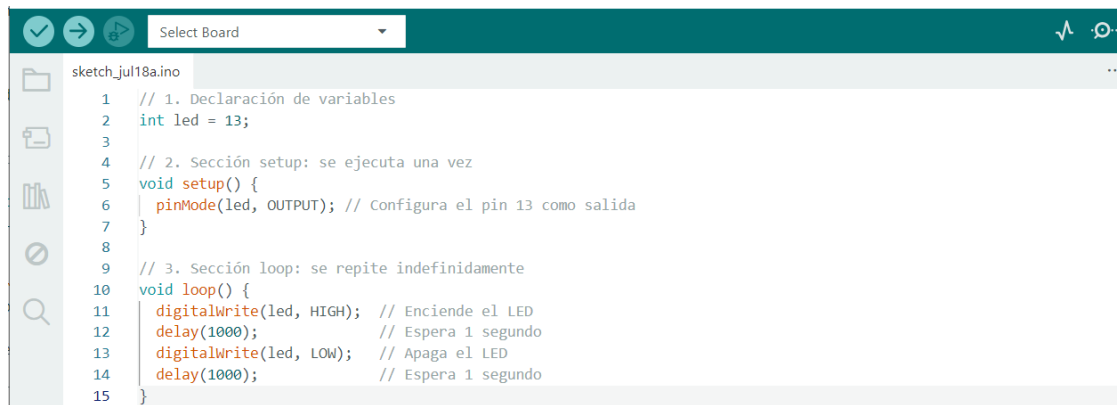


Imagen 3.5.2- Sketch de Ejemplo.

Con Arduino se pueden realizar multitud de proyectos de rango muy variado: desde robótica hasta domótica, pasando por monitorización de sensores ambientales, sistemas de navegación, telemática, etc. Las posibilidades de esta plataforma para el desarrollo de productos electrónicos son prácticamente infinitas y tan solo están limitadas por nuestra imaginación.

En nuestro caso, decidimos dedicar una parte significativa de nuestras prácticas al trabajo con Arduino. Los proyectos desarrollados se describen en detalle en la sección 4.5.

3.6 Python

Python es un lenguaje de programación de alto nivel, interpretado y de código abierto, creado por Guido van Rossum y lanzado en el año 1991. Su principal característica es la simplicidad de su sintaxis, ya que permite escribir código de forma clara y concisa, lo que lo hace accesible tanto para principiantes como para profesionales.

Algunos de los principales usos de Python son los siguientes:

- Desarrollo web.
- Automatización de tareas y creación de flujos de trabajo.
- Conexión con bases de datos y manipulación de archivos.
- Análisis de datos y cálculos matemáticos complejos.
- Desarrollo rápido de prototipos o software final.

En nuestros últimos días en el centro, Miguel nos impartió conocimientos básicos sobre Python. Se mostrarán los ejercicios realizados en la sección 4.6 del documento.

4. Proyectos Realizados.

4.1 Proyecto con Postman

Durante nuestra primera semana, Miguel nos introdujo a los conceptos básicos de la arquitectura FIWARE y al uso de la herramienta Postman, como ya vimos en la sección 3.1.1. Como ejercicio práctico, nos propuso desarrollar un pequeño proyecto que integrara ambos elementos, que consistiría en diseñar una Smart City simulada mediante la sensorización de un entorno para su posterior gestión.

La sensorización en este caso sería hipotética, es decir, nosotros definiríamos las entidades y atributos necesarios para representar digitalmente el entorno elegido, simulando así el funcionamiento de una ciudad inteligente sin necesidad de sensores físicos reales.

Decidimos elegir un aeropuerto como entorno para nuestro proyecto porque es un espacio que dispone de muchas áreas y elementos diferentes que pueden ser gestionados y monitorizados digitalmente. Además, no solo cuenta con terminales y pistas, sino también tiendas, vuelos, personal y otras muchas partes que se pueden representar con entidades y atributos en FIWARE.

Por otro lado, en un aeropuerto se necesitan gestionar muchos datos en tiempo real, como las condiciones meteorológicas, el estado de las pistas, la afluencia de personas o los horarios de los vuelos. Es por ello que pensamos que sería un escenario perfecto para aplicar suscripciones y recibir notificaciones automáticas con el Context Broker de FIWARE.

También nos interesaba porque nos permite trabajar con estructuras jerárquicas y referencias entre entidades, por ejemplo, relacionar una pista con una terminal o un vuelo con una puerta de embarque. De esta forma, planteamos dicho diseño jerárquico, donde el aeropuerto actúa como entidad principal, y cada terminal como un entorno contenido. A su vez, cada terminal incluye entidades correspondientes a establecimientos comerciales, pistas de aterrizaje, aseos, mobiliario, etc.

La entidad del aeropuerto posee atributos como la afluencia de personas o el estado (si está abierto o cerrado), este último haciendo uso de un booleano que será “true” si lo está y “false” si no. Asimismo, cada terminal tiene una estructura similar, y se relaciona con el aeropuerto mediante una referencia (*relationship*) y se le asignan sus propios atributos como estado (*abierto/cerrado*) y afluencia de personas. Podemos ver todo esto en las **Imágenes 4.1.1 y 4.1.2**.

```

1  {
2      "id": "aeropuerto001",
3      "type": "aeropuerto",
4      "Abierto": {
5          "value": false,
6          "type": "Boolean"
7      },
8      "numeroDePersonas": {
9          "value": 0,
10         "type": "number"
11     }
12 }
13 }
14

```

Imagen 4.1.1 – Entidad Aeropuerto.

```

1  {
2      "id": "terminal001",
3      "type": "terminal",
4      "Abierto": {
5          "value": false,
6          "type": "Boolean"
7      },
8      "numeroDePersonas": {
9          "value": 0,
10         "type": "number"
11     },
12     "refAeropuerto": {
13         "type": "Relationship",
14         "value": "aeropuerto001"
15     }
16 }
17

```

Imagen 4.1.2 – Entidad Terminal con referencia a Aeropuerto.

Posteriormente, desarrollamos entidades secundarias vinculadas a cada terminal, como por ejemplo las pistas de aterrizaje, que cuentan con sensores que reflejan condiciones meteorológicas que pueden afectar a los vuelos (lluvia, viento, niebla).

También creamos entidades para los vuelos, asociadas a cada pista, que contienen información relevante como: aerolínea, destino, hora de salida, hora de llegada, número de pasajeros y puerta de embarque, entre otros atributos.

Además, incluimos entidades correspondientes a los establecimientos y servicios disponibles dentro de cada terminal, como restaurantes, tiendas y aseos. Estas disponen de atributos relacionados con su estado de mantenimiento, disponibilidad y afluencia de personas, simulando un sistema de monitorización en tiempo real.

Una vez más, para reflejar correctamente la estructura jerárquica del entorno, utilizamos relaciones entre entidades (referencias), lo cual permite mantener una conexión clara entre los diferentes niveles del modelo (por ejemplo, vuelos vinculados a pistas, y estas a su vez a terminales).

En la **Figura 4.1.1** se muestra un esquema gráfico representando la estructura de una de las terminales, incluyendo sus entidades, atributos y relaciones.

Con las entidades ya creadas, implementamos también suscripciones, que nos permiten monitorizar los cambios en atributos específicos. Por ejemplo, al cambiar el valor de “Libre” en una pista de aterrizaje, se envía una notificación a un webhook configurado previamente, obteniendo una notificación en tiempo real de las actualizaciones.

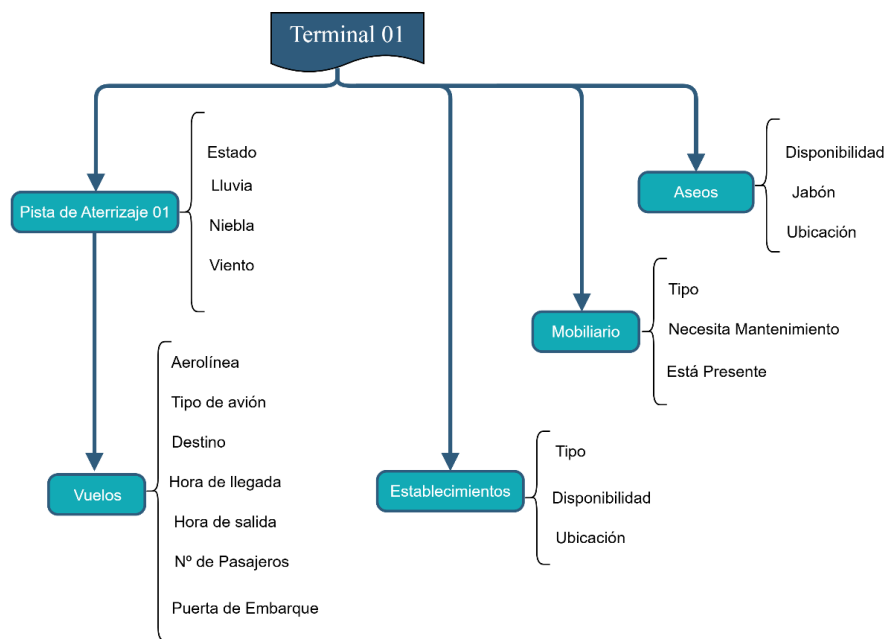


Figura 4.1.1 – Esquema del Proyecto.

4.2 Alarma con Node-RED

Para abrir Node-RED una vez lo tengamos instalado, debemos escribir el comando *node-red* en el terminal. Posteriormente, para abrir la interfaz gráfica debemos escribir en la barra de direcciones la siguiente URL: <http://localhost:1880>.

Para familiarizarnos con Node-Red, decidimos hacer una pequeña alarma sonora que anunciase el número de vuelo. La idea era que, al introducir un número de vuelo y pulsar un botón, el sistema pronunciara automáticamente una frase con ese valor.

Para ello, empezamos con un nodo *inject*, donde configuramos directamente el número de vuelo como texto. Este nodo lo conectamos a un botón (*button*) del dashboard, que actúa como disparador

manual. A continuación, añadimos un nodo *template*, donde escribimos el mensaje con el siguiente formato:

Vuelo: {{payload}} Aterriza en Pista 4!

La palabra {{payload}} se reemplaza automáticamente por el número de vuelo que hayamos puesto en el nodo *inject*, construyéndose así el mensaje final que queremos escuchar.

Por último, conectamos el flujo a un nodo audio *out*, que se encarga de reproducir el mensaje en voz alta.

Para probarlo, simplemente abrimos la dirección <http://localhost:1880/ui>, y al pulsar el botón del flujo, se reproduce la frase generada por el sistema.

En la **Imagen 4.2.1** se muestra el flujo de nodos utilizado.

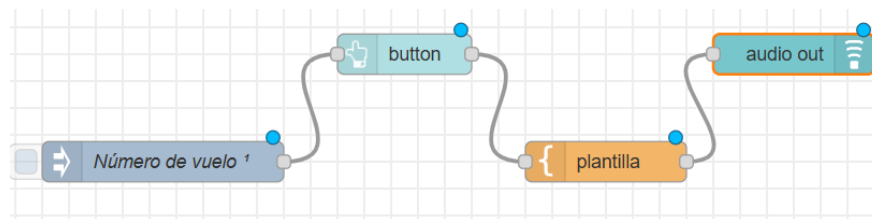


Imagen 4.2.1 – Alarma Vuelo.

4.3 Proyecto con KNIME

Para introducirme en Knime, decidí realizar un proyecto basado en segmentación, una técnica muy importante en el análisis de datos que permite dividir un conjunto de datos en grupos o también llamados *clústeres*, agrupando elementos que comparten características similares.

Existen dos grandes enfoques de segmentación: jerárquica y no jerárquica. En mi caso trabajé con segmentación no jerárquica, que parte de un número de grupos predefinido, y va clasificando cada dato en el grupo con el que más se parece, basándose en distancias. A diferencia de la segmentación jerárquica (que crea árboles de agrupación sin conocer de antemano el número de grupos), esta busca esos grupos desde el principio y los va refinando iterativamente. Es muy útil cuando queremos descubrir patrones o perfiles ocultos en los datos, sin necesidad de conocer etiquetas previas.

Comencé descargando un archivo en formato .csv con datos (en mi caso, datos relacionados con consumidores de un centro comercial). Lo cargué en KNIME usando el nodo *CSV Reader* y se transforma en un archivo Excel mediante *Excel Writer*, simplemente para trabajar de forma más cómoda.



Imagen 4.3.1 – Importación del archivo a .xlsx.

Una vez importado en formato .xlsx, lo conecté a un nodo *Column Filter* para eliminar columnas no necesarias. Después, utilicé el nodo *Category to Number* para convertir las variables categóricas (como nombres, género, país, etc.) en valores numéricos, necesarios para poder operar

matemáticamente con ellas. Esta conversión fue verificada usando un nodo *GroupBy*, el cual nos permite comprobar qué número se asignó a cada categoría.

Antes de aplicar el algoritmo de segmentación, hay que normalizar los datos con el nodo *Normalizer*, ajustando todos los valores a un mismo rango (entre 0 y 1), ya que esto evita que algunas variables dominen sobre otras por tener valores más altos.

Cabe destacar que, antes de cargar la base de datos en el primer nodo, se debe haber revisado que no haya errores, como datos atípicos o espacios vacíos (esto se conoce como limpiar base de datos).

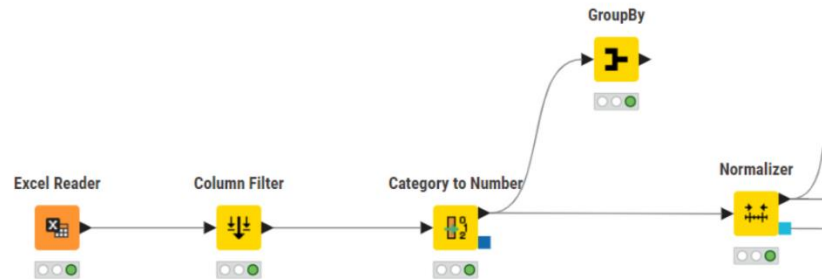
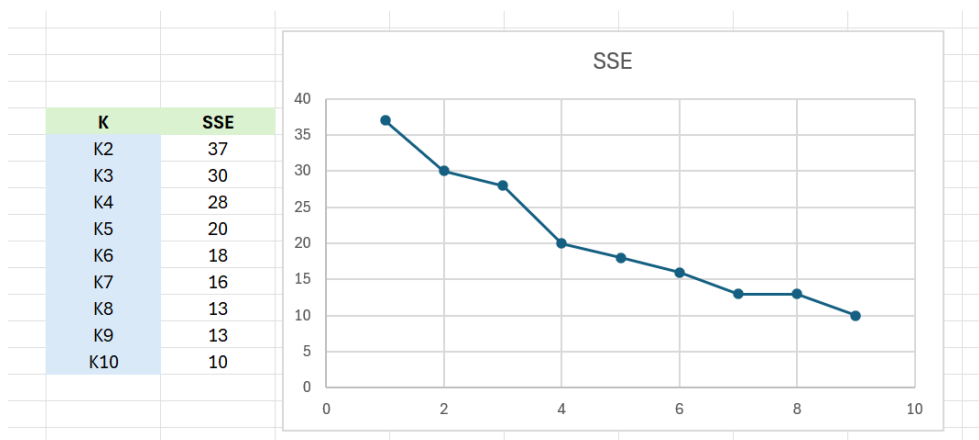


Imagen 4.3.2 – Normalización.

Con los datos ya preparados y normalizados como se muestra en la **Imagen 4.3.2**, procedí a aplicar el algoritmo K-Means. Este es uno de los métodos de segmentación más utilizados en ciencia de datos, cuyo objetivo principal es dividir un conjunto de datos en "K" grupos (o clústeres como mencionamos anteriormente), de manera que los elementos dentro de un mismo grupo sean lo más parecidos posible entre sí, y a la vez, lo más diferentes posible de los elementos de los otros grupos. Para ello, se usan los nodos *SimpleKMeans* (para hacer pruebas iniciales con distintos números de clústeres) y *K-Means* (para aplicar el modelo final).

Para determinar cuál era el número óptimo de grupos (valor de K), ejecuté el algoritmo varias veces probando valores de 2 a 10. Cada ejecución nos da un valor de suma de cuadrados, que refleja qué tan compactos son los grupos formados. Estos valores los anoté en una tabla y posteriormente los grafiqué en Excel. Para elegir el K óptimo se puede usar, entre otros, el “método del codo” (Elbow Method). En la gráfica nos vamos a encontrar que la curva baja mucho al principio y luego empieza a disminuir notablemente, el punto donde ocurre esto es el “codo” y suele ser el K óptimo. En mi caso (**Gráfica 4.3.1**), este punto corresponde con $K=4$, así que usé ese valor para aplicar el modelo final.



Gráfica 4.3.1 – Elbow Method.

A partir de ahí, se conecta el nodo de normalización al nodo *K-Means*, introduciendo el número de clúster (4), y luego conecté todo al nodo *Denormalizer* para volver a la escala original de los datos y poder analizarlos de forma más clara.

Una vez aplicados los clústeres, utilicé algunos nodos para evaluar la calidad del modelo:

- **Coeficiente de Silueta:** mide cómo de definido está cada grupo. Cuanto más alto, mejor diferenciados están los clústeres.
- **Entropía:** indica cuán homogéneo es cada grupo.
- **Curvas ROC:** el nodo ROC Curve proporciona un gráfico por clúster que nos indica qué tan útiles son las variables para identificar ese grupo.

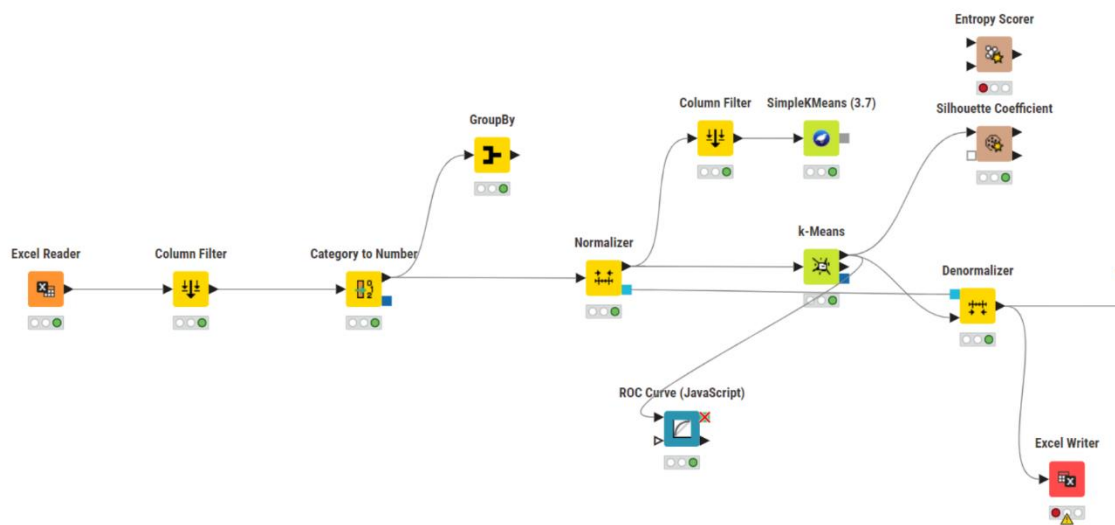
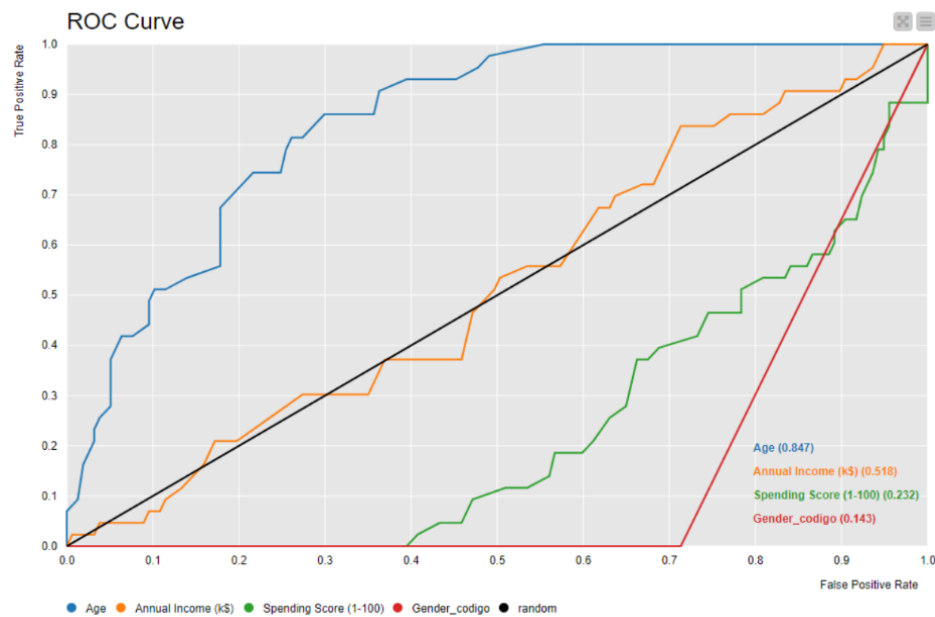


Imagen 4.3.3 – Algoritmo K-Means.

Cada curva ROC representa la relación entre la tasa de verdaderos positivos (eje Y) y la tasa de falsos positivos (eje X). Cuanto más se aproxime la curva al vértice superior izquierdo del gráfico, mejor será el rendimiento de esa variable como discriminadora de un clúster concreto. Es decir, una buena variable tendrá una alta tasa de aciertos y pocos errores al clasificar.

Por ejemplo, en nuestro modelo se puede observar en la **Gráfica 4.3.2** que la variable *Age* tiene una curva con un área superior a 0.7, lo cual la convierte en un buen predictor para diferenciar los grupos. Por otro lado, *Annual Income* apenas supera el 0.5, lo que indica que no aporta demasiada información a la segmentación. Otras variables, con valores más cercanos a 0.4 o inferiores, resultan ser poco representativas o incluso irrelevantes en el proceso de agrupación.



Gráfica 4.3.2 – Curva ROC.

Una vez finalizada la segmentación mediante el algoritmo K-Means y validada la calidad de los clústeres, pasamos a la fase descriptiva, que nos permite interpretar de forma más detallada las características de cada grupo.

Para ello, exporté los resultados a Excel y construí varias tablas dinámicas con el objetivo de analizar y comparar los clústeres generados. En la primera tabla, representé la distribución de género en cada grupo, colocando el género en las filas y los clústeres en las columnas. Esto nos permite observar si existe alguna concentración específica de hombres o mujeres en determinados segmentos.

Posteriormente, creé una segunda tabla para calcular el promedio de edad dentro de cada clúster. Esta información es útil para entender el perfil demográfico de los distintos grupos formados. Además, elaboré otras tablas para estudiar el ingreso anual, calculando tanto el promedio como la desviación estándar, que nos permite evaluar la variabilidad del ingreso dentro de cada grupo.

Cuenta de Age	Etiquetas de columna				
Etiquetas de fila	cluster_0	cluster_1	cluster_2	cluster_3	Total general
Female	100%	0%	0%	0%	56%
Male	0%	100%	100%	100%	44%
Total general	100%	100%	100%	100%	100%
	Etiquetas de columna				
	cluster_0	cluster_1	cluster_2	cluster_3	Total general
Promedio de Age	38,10	24,04	52,44	33,28	38,85
	Etiquetas de columna				
	cluster_0	cluster_1	cluster_2	cluster_3	Total general
Promedio de Annual Income (k\$)	59,25	45,96	62,02	87,11	60,56
	Etiquetas de columna				
	cluster_0	cluster_1	cluster_2	cluster_3	Total general
Desvest de Annual Income (k\$)	26,01	20,55	25,06	18,93	26,26

Tabla 4.3.1 – Tabla Descriptiva de los Datos.

4.4 Evento en Olivenza.

Durante nuestra estancia en FIWARE Space, tuvimos la oportunidad de participar en un evento que tuvo lugar en un Fab Lab de Olivenza. Este es uno de los cuatro espacios que conforman el Centro de Innovación Territorial (CITLabs) en la provincia, puesto en marcha por la Diputación de Badajoz con ayuda del Ministerio.

Los Laboratorios de Fabricación Digital (Fab Lab) son espacios multidisciplinares que cuentan con herramientas digitales como impresoras 3D, fresadoras de cnc y maquinaria de corte láser, y que se encuentran abiertos a todo tipo de personas, ya sean estudiantes, trabajadores o emprendedores que quieran aprender o realizar proyectos con este tipo de tecnologías.

Este es un centro abierto donde con previa inscripción gratuita se pueden recibir cursos, talleres prácticos o programas para todas las edades sobre el funcionamiento de máquinas y elaboración de proyectos.

No solo fomenta la innovación, sino que también es un punto de encuentro para el intercambio de conocimientos y la colaboración, como es nuestro caso, pues asistimos a un taller sobre la solución tecnológica *Respira*. Este es un proyecto de código abierto compuesto por sensores que miden la calidad del aire y recogen dicha información en una plataforma web donde se analizan el estado del aire que respiramos. Existen multitud de sensores distribuidos por todo el territorio, de forma que en la web podemos clicar sobre uno de ellos y ver la calidad del aire en dicha localización.

Nos enseñaron a montar uno de los mencionados sensores de calidad del aire desde cero, usando componentes electrónicos básicos y una placa compatible con Arduino.

Siguiendo las instrucciones, montamos nuestro propio dispositivo, que posteriormente fue conectado y subido a la plataforma de *Respira* (<http://calidadmedioambiental.org/dashboard>).

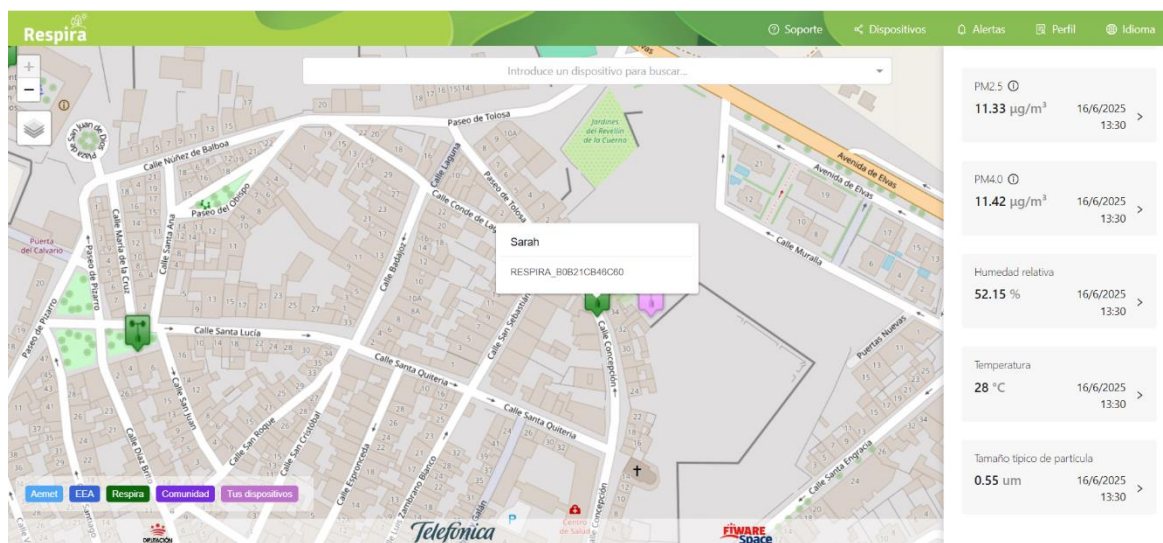


Imagen 4.4.1 – Visualización en la plataforma web del sensor fabricado.

Como se observa en la **Imagen 4.4.1**, podemos ver la información de mi sensor en el margen izquierdo de la página; y en el resto del mapa, si nos alejamos, se mostrarán todos los demás sensores por la zona con un icono verde, sobre el que podremos pinchar para ver las características del aire en dichas zonas.

Entre los sensores y elementos que empleamos se encuentran:

- **Sensor SPS30 (Sensirion):** mide la concentración de partículas en suspensión PM1.0, PM2.5, PM4 y PM10.
- **Sensor DHT22:** capta temperatura y humedad relativa.
- **Sensor multigás Grove (SGP30):** permite medir compuestos orgánicos volátiles (VOC) y dióxido de carbono equivalente (eCO₂).
- **Placa de desarrollo con microcontrolador ESP32:** permite enviar los datos a internet gracias a su conectividad Wi-Fi.



Imagen 4.4.2 – Sensor SPS30.

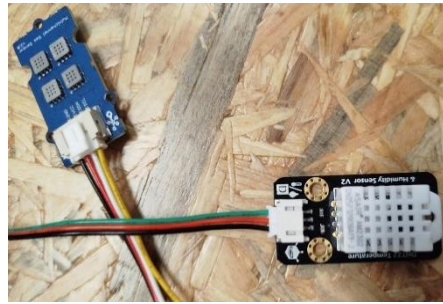


Imagen 4.4.3 – Sensores DHT22 (blanco) y multigás (azul).

Una vez realizado el montaje físico, cargamos el código en la placa con Arduino IDE, conectamos los sensores, y verificamos la lectura de los datos. Finalmente, el dispositivo comenzó a transmitir datos a la nube, que se podían consultar públicamente en el portal web mencionado anteriormente. En las siguientes imágenes se muestra el sensor ya montado y en un encapsulado hecho con impresora 3D.



Imagen 4.4.4 – Parte Frontal del Sensor.



Imagen 4.4.5 – Parte Posterior del Sensor.

4.5 Proyecto Final con Arduino

Para aprender a utilizar mejor Arduino decidimos usarlo como herramienta para nuestro proyecto final. Como ya habíamos planteado el Smart City de un Aeropuerto en el proyecto de FIWARE, pensamos en construirlo sobre esta idea. Simulamos la automatización del aeropuerto con sensores para optimizar la gestión de hangares, evitar colisiones entre aviones en cruces y mejorar la seguridad en el despegue mediante el análisis de condiciones atmosféricas. A continuación, se expone un breve resumen técnico del proyecto:

Control de hangares disponibles:

Descripción:

Haciendo uso de sensores que detectan obstáculos, podemos detectar si un hangar está libre u ocupado por un avión, iluminando un LED según corresponda:

- Si el sensor detecta que hay un avión, se enciende un LED rojo indicando que el hangar está ocupado.
- Si no hay avión, se enciende un LED verde, indicando disponibilidad.

Materiales:

- En primer lugar, necesitamos una **placa Arduino**, en nuestro caso **Arduino Uno R4 WiFi** junto con un **cable USB compatible** para conectar al PC y una **protoboard** para realizar las conexiones.
- **Sensor Infrarrojo (IR):** Este dispositivo detecta la presencia de un objeto mediante la reflexión de luz infrarroja. Está formado por un LED emisor de luz infrarroja y un fotodiodo que recibe la luz reflejada por el obstáculo.
- **LED RGB:** Contiene los colores primarios verde, rojo y azul, de forma que mediante la adición de ellos se pueden obtener el resto de los colores. Nos interesa este tipo de LED porque podemos integrar el verde y el rojo en el mismo dispositivo, en vez de utilizar dos LEDs.
- **3 resistencias de 220 Ω .**

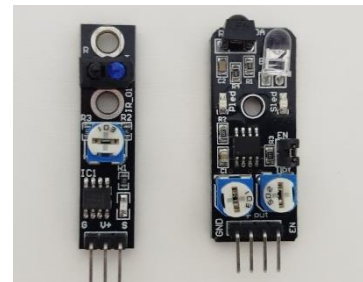


Imagen 4.5.1 – Sensores IR.

Montaje:

El montaje es bastante sencillo, como se muestra en **Imagen 4.5.3**, basta con conectar el módulo a los pines de 5V y GND de Arduino. La salida del sensor se conecta a un pin digital, en nuestro caso al 4 y añadimos un LED RGB de cátodo común con tres resistencias de 220 Ω . En el diagrama mostrado en **Imagen 4.5.2**, como el programa no disponía de sensor IR, se ha sustituido por un sensor PIR de movimiento.

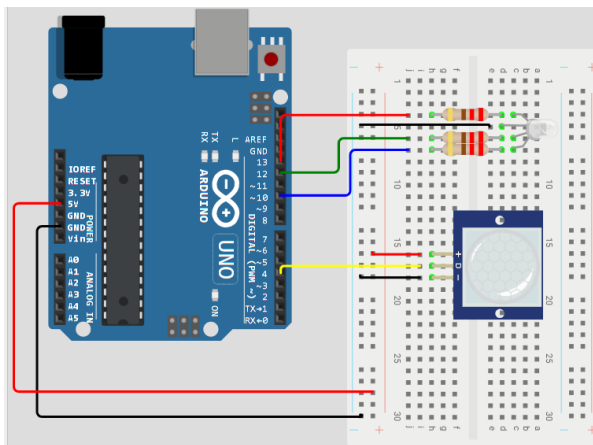


Imagen 4.5.2 – Diagrama del montaje.

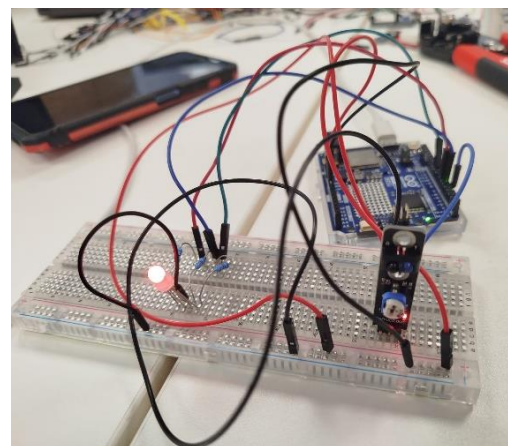


Imagen 4.5.3 – Montaje final.

Control de cruce:

Descripción:

Este módulo consiste en utilizar dos sensores de presencia en un cruce, uno en cada carril, para detectar la presencia de los aviones. Cuando ambos sensores detecten un avión en un determinado periodo de tiempo en cada carril, se manda una alerta de peligro de colisión. Haremos uso también de un zumbador para la alarma.

Materiales:

- En este caso, de nuevo usaremos la **placa Arduino**, **cable de conexión** y **protoboard**. Además, necesitaremos **dos sensores IR** y **dos LEDs RGB**, uno para cada carril.
- **Zumbador**: es un dispositivo que contiene en su interior un material piezoeléctrico, de forma que al aplicarle una señal eléctrica se deforma produciendo sonido. En nuestro caso, será útil para crear la alarma de alerta.

Montaje:

Procedemos de forma similar al anterior caso, solo que esta vez utilizamos dos sensores IR y dos LED. Simularemos que cada uno está en un carril distinto y cuando ambos detecten un avión, deben encenderse los LEDs.

De nuevo, se han utilizado sensores PIR para sustituir los sensores IR en el diagrama de **Imagen 4.5.4**.

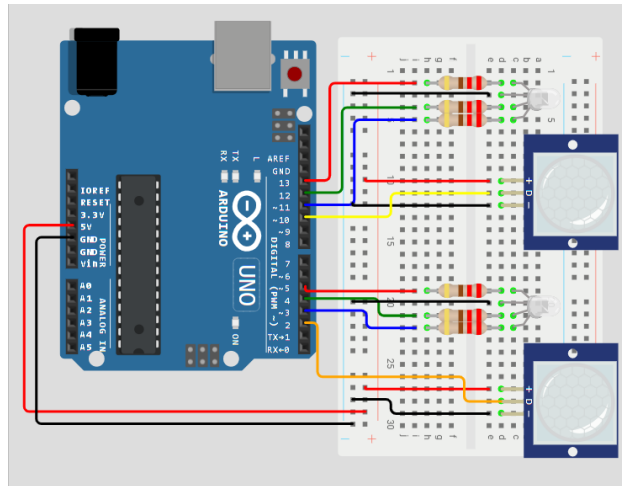


Imagen 4.5.4 – Diagrama del montaje.

En la **Imagen 4.5.5** se muestra el montaje realizado en el Centro. Se tapa uno de los sensores IR (izquierdo) y el led se turna rojo (en la imagen los LEDs están alternados, el de la derecha corresponde al sensor de la izquierda). El otro sensor (derecha), al no recibir señal de obstáculo, el LED se enciende verde.

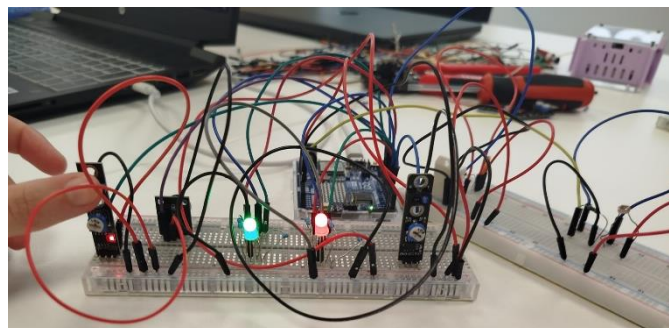


Imagen 4.5.5 – Montaje final.

Control de niebla y condiciones atmosféricas:

Descripción:

Con el uso de un sensor de temperatura y humedad y otro de luminosidad, podemos detectar si hay niebla en la pista y si las condiciones son favorables o no para el despegue de los aviones, con el fin de aumentar la seguridad en los despegues.

Materiales:

- **Placa Arduino, cable de conexión y protoboard.**
- **LED RGB.**
- **Sensor DHT:** es un sensor digital que mide temperatura y humedad.
- **Sensor de luminosidad LDR:** es un fotorresistor cuyo valor de resistencia es inversamente proporcional a la intensidad de luz que se le aplique. De esta forma puede detectar la luminosidad del entorno.

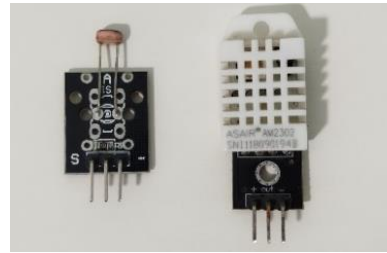


Imagen 4.5.6 – Sensor LDR y Sensor DHT12.

Montaje:

El montaje no es más que conectar el DHT y el LDR a la placa y con un LED RGB que se encenderá según haya niebla (rojo) o no (verde).

Para la detección de niebla, realizamos un programa en el que, si el DHT detecta humedad muy alta (aprox > 85%) y el LDR luminosidad baja, es que hay niebla.

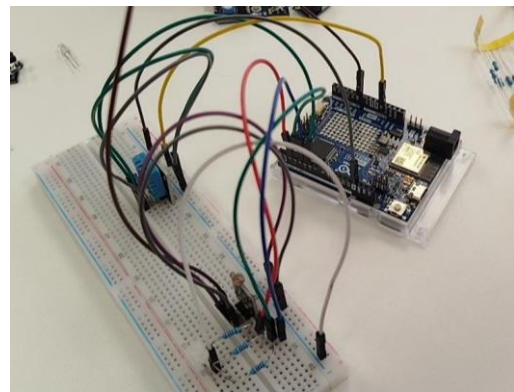


Imagen 4.5.8 – Montaje final.

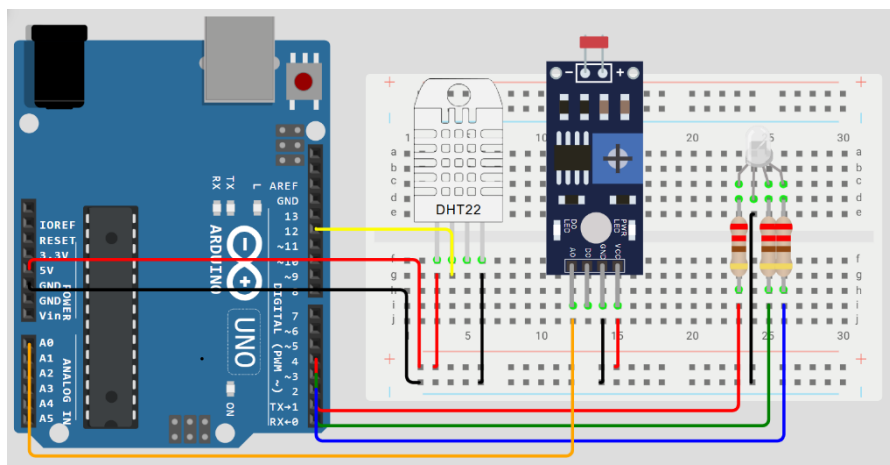


Imagen 4.5.7 – Diagrama del montaje.

Integración final:

Tras desarrollar y probar cada módulo de forma independiente, integramos todos en un mismo programa, organizando el código y ajustando el montaje a una misma placa Arduino.

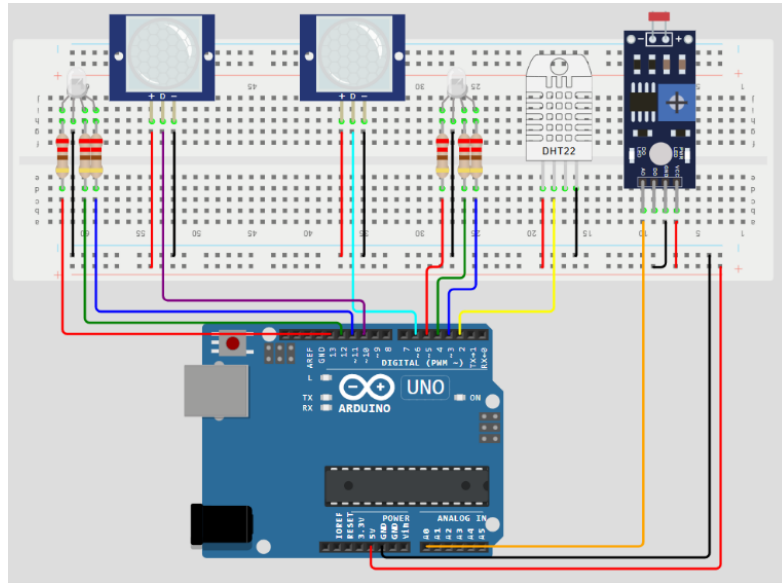


Imagen 4.5.9 – Diagrama de la Integración Final.

Una vez finalizado el montaje físico, procedimos a conectar nuestra maqueta del aeropuerto inteligente con una plataforma IoT externa. Gracias a la placa Arduino Uno R4 WiFi pudimos conectar el proyecto a internet y enviar los datos en tiempo real a FIWARE. En este punto el sistema ya es capaz de:

- Detectar la disponibilidad de los hangares mediante sensores de presencia, mostrando el estado con luces LED.
- Alertar de posibles cruces peligrosos entre aeronaves, mediante la lectura simultánea de sensores en ambos carriles y la activación de luces y alarmas.
- Analizar las condiciones atmosféricas, detectando niebla en base a la humedad ambiental y la intensidad de luz captada por un sensor.

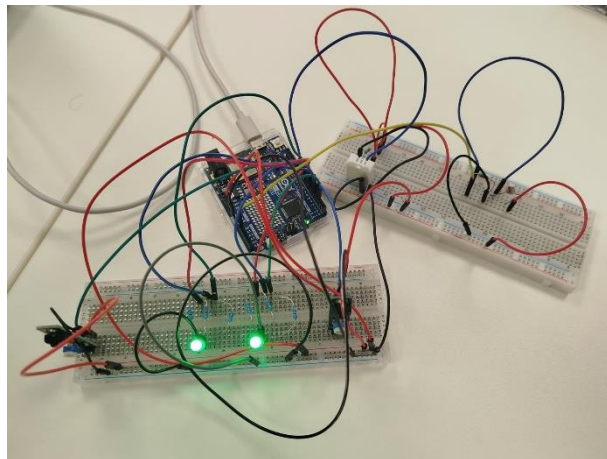


Imagen 4.5.10 – Montaje Integración Final.

Con el programa los datos recopilados por los sensores fueron enviados periódicamente a la nube, utilizando peticiones HTTP. Esto permitió que toda la información del entorno (presencia, humedad, temperatura, visibilidad, etc.) estuviera disponible remotamente, abriendo paso a una posible integración con plataformas de gestión más completas.

Con esta integración, logramos simular un entorno aeroportuario inteligente, donde los sensores no solo actúan localmente, sino que también permiten tomar decisiones a partir de datos en la nube, representando así una visión más realista del concepto de Smart City.

4.6 Python

Comenzamos haciendo ejercicios para familiarizarnos con la lógica de la programación y estructura del lenguaje. Miguel nos explicó los conceptos básicos, centrándose en las estructuras de datos más comunes como las listas y los diccionarios.

Empezamos con ejercicios sencillos, como crear una lista de números e ir generando una segunda lista cuyos elementos fuesen la suma acumulada de los elementos de la primera. El ejercicio resuelto se muestra en la **Imagen 4.6.1**, donde primero creé una lista inicial (*lista1*) con elementos y otra segunda lista vacía (*lista2*). Usé un bucle que recorre cada número de la *lista1*, sumándolos progresivamente con una variable *sum* y utilizando la función *append()* para ir añadiendo cada nuevo valor a *lista2*. Al final, también utilizamos funciones como *len()* para obtener la longitud de una lista e *index()* para encontrar la posición de un elemento concreto.

```
3
4 #EJERCICIO 1
5
6 lista1=[1,2,3,4] #Creamos una lista
7 lista2=[] #Lista en blanco
8
9 sum=0
10 for i in lista1:
11     sum=sum+i
12     lista2.append(sum)
13 print(lista2)
14
15 l=len(lista2) #Longitud de la lista
16 print(l)
17
18 element=3
19 print(lista1.index(element)) #Posicion que ocupa el elemento
20
```

Imagen 4.6.1 – Primer Ejercicio.

En el siguiente ejercicio, trabajamos con cadenas de texto. El objetivo era contar el número de palabras en un texto introducido por el usuario. Para ello, usé un *input()* para recoger el texto, y luego la función *.split()*, que lo divide automáticamente en palabras separadas por espacios, generando una lista. Además, con *len()* contamos cuántas palabras había.

En la segunda parte, añadimos la posibilidad de buscar una palabra concreta dentro del texto. Usamos otro *input()* para que el usuario introdujera la palabra a buscar, y con un *bucle for*, recorremos la lista de palabras buscando coincidencias. De esta forma, si encuentra la palabra, imprime su posición utilizando *index()*.

```

1 # a) Pasar un párrafo y devolver el numero de palabras:
2
3 texto=input("Dame un párrafo")
4 palabras=texto.split()           #Con .split() se separa el texto por espacios
5 print(len(palabras))
6
7
8 # b) Detectar si una palabra en concreto esta en el texto, y si lo esta, en que posicion:
9
10 palabra=input("Dime que palabra quieres detectar en el texto")
11
12 #Bucle con cada elemento de la lista. Si el elemento coincide con la palabra que se busca, se da la posicion:
13 for i in palabras:
14     if i==palabra:
15         print("La palabra está en el texto")
16         posicion=palabras.index(palabra) + 1 #posicion que ocupa el elemento
17         print("Su posición es:" + " " + str(posicion))
18

```

Imagen 4.6.2 – Segundo Ejercicio.

Más adelante, comenzamos a trabajar con diccionarios, una estructura que permite almacenar pares de *clave-valor*. Para familiarizarme con la sintaxis, estuve practicando cómo crear diccionarios, acceder a sus elementos, modificarlo e iterarlos, como se muestra en Imagen 4.6.3.

```

1 Diccionario1= {"Nombre": "Sarah", "Apellido":"Gonzalez", "Ciudad":"Badajoz" }
2 print(Diccionario1)
3 Diccionario1['Nombre']="Maria"      #Para modificar un elemento usamos []. Si no esta, se añade
4 print(Diccionario1)
5
6 #Itinerar diccionario
7 for i in Diccionario1: #Imprime los key de diccionario
8     print(i)
9
10 for i in Diccionario1: #Imprime los valores del diccionario
11     print(Diccionario1[i])
12

```

Imagen 4.6.3 – Diccionarios.

Una vez comprendida la base, realizamos un ejercicio más complejo: crear un programa que, dado un párrafo, genere un diccionario con todas las palabras que contiene y cuente cuántas veces aparece cada una.

Primero, pedimos al usuario un texto con `input()`. Luego, lo convertimos en lista usando `.split()` para separar palabra por palabra. A partir de esa lista, creamos un diccionario vacío donde, a través de un bucle, vamos contando la frecuencia de cada palabra. El resultado es un diccionario en el que las claves son las palabras y los valores indican cuántas veces aparece cada una.

```

13
14 #Programa que saque diccionario con todas las palabras del parrafo y nos diga la frecuencia (numero de veces que sale en el parrafo)
15 #Primero pasamos el texto a una lista y contamos las veces que se repite cada palabra
16
17 texto = input("Dame un párrafo: ")
18 lista = texto.split()
19 Dic = {}
20
21 for i in lista:
22     if i in Dic:
23         Dic[i] += 1 # si ya existe, suma 1
24     else:
25         Dic[i] = 1 # si no existe, la añade con valor 1
26
27 print(Dic)

```

Imagen 4.6.4 – Tercer Ejercicio.

Posteriormente, Miguel nos comentó sobre el uso de librerías externas en Python. Estas son un conjunto de funciones y clases ya programadas que podemos reutilizar para tareas concretas, como crear documentos, trabajar con datos, generar gráficos, etc. Algunas de las más utilizadas son:

- **NumPy:** Utilizada para cálculos numéricos y álgebra lineal. Es muy útil para realizar operaciones matemáticas avanzadas y trabajar con proyectos donde se manejan grandes volúmenes de datos numéricos o cálculos científicos complejos.
- **Pandas:** Esta librería es una de las más usadas en análisis de datos ya que permite trabajar con datos estructurados, especialmente tablas (como en Excel o CSV). Además, tiene herramientas para limpiar, filtrar, agrupar y transformar datos de forma eficiente.
- **Matplotlib:** Nos permite crear gráficos y representar de forma visual los datos analizados y encontrar patrones o tendencias.
- **Scikit-learn:** Esta librería contiene algoritmos listos para usar en tareas como clasificación, regresión y clustering (agrupación), como el algoritmo K-Means que usamos en los ejercicios de segmentación. Es ideal para empezar en el mundo de la inteligencia artificial.

En nuestro caso, estuvimos trabajando con la *librería FPDF*, una herramienta muy útil que permite generar archivos PDF desde código Python, sin tener que utilizar Word u otros editores. A continuación, se muestra el código que desarrollamos como ejemplo para probar la librería.

```
1 #Libreria fpdf:
2
3 from fpdf import FPDF
4 class PDF(FPDF) :          # Definimos una subclase
5     def header(self):
6         # Rendering logo:
7         #self.image("../docs/fpdf2-logo.png", 10, 8, 33)
8         # Setting font: helvetica bold 15
9         self.set_font("helvetica", style="B", size=15)
10        # Moving cursor to the right:
11        self.cell(80)
12        # Printing title:
13        self.cell(30, 10, "Title", border=1, align="C")
14        # Performing a line break:
15        self.ln(20)
16
17 pdf=FPDF(orientation="P", unit="mm", format="A4") # Orientacion: vertical (p), horizontal(L). Formato (Letter,Legal),. Unidades(pt,cm,in)
18 pdf.add_page() # Añadimos una pagina con add_page
19 pdf.set_font("Times", style="B", size=12) # Tipo de fuente, tamaño y estilo(negrita, cursiva(I), subrayado(U))
20 pdf.cell(40,10,"Hola Mundo",1) # Texto entre recuadro
21 pdf.output("Ejercicio4.pdf")
```

Imagen 4.6.5 – Ejercicio con Librería FPDF.

5. Conclusión

A lo largo de mi paso por el centro durante las 100 horas de prácticas, he tenido la oportunidad de trabajar con herramientas y tecnologías que hasta ahora solo conocía de forma superficial o teórica, como Arduino o Python y también con otras que desconocía por completo, como FIWARE, Postman o Knime.

Además de entender cómo funcionan, he aprendido a integrarlas en proyectos, a resolver problemas técnicos por mí misma y a organizarme en un entorno de trabajo más cercano al mundo laboral que al universitario. Cada uno de los proyectos realizados, desde la simulación de una Smart City hasta la creación de sensores con Arduino o la segmentación de datos en KNIME, me han ayudado a adquirir habilidades y capacidad de adaptación para el futuro.

Más allá de lo técnico, estas prácticas también me han permitido trabajar en equipo, especialmente con mi compañero y amigo Javier, y conocer a grandes personas como mis tutores Jorge y Miguel. La experiencia me ha enseñado a desenvolverme con herramientas nuevas en poco tiempo y a comprender mejor cómo se planifican y ejecutan los proyectos en un entorno real.

En definitiva, considero que estas prácticas han sido una experiencia muy positiva que me han aportado una visión más completa y aplicada del sector tecnológico, y han reafirmado mi interés por seguir formándome en este ámbito.

6. Bibliografía

- [1] Arduino. *Arduino Documentation* [en línea]. Dirección URL: <https://docs.arduino.cc/>. [Consulta: 13 junio 2025].
- [2] CURSO KNIME ESPAÑOL. *Tutorial en vídeo*. [vídeo en línea]. Dirección URL: https://www.youtube.com/playlist?list=PLYqPCyWfFHmb-Bb5nBrDBQF_h73ATdll7. [Consulta: 10 junio 2025].
- [3] Diputación de Badajoz. *Olivenza FabLab Documentation* [en línea]. Dirección URL: <https://dip-badajoz.es/agenda/tyd/index.php?id=3&agenda=21792&3=0&4=40&7=2>. [Consulta: 16 junio 2025].
- [4] TORRENTE ARTERO, O. 2013. *Arduino. Curso práctico de formación*. Primera Edición. México: Alfaomega Grupo Editor, S.A. de C.V. [Consulta: 13 junio 2025].
- [5] FIWARE Foundation. *FIWARE Documentation* [en línea]. Dirección URL: <https://fiware.org/developers/>. [Consulta: 3 junio 2025].
- [6] KNIME. *Beginner's Luck: Guía para empezar con KNIME v4.4* [en línea]. Dirección URL: https://www.knime.com/sites/default/files/2022-12/KNIME_Beginner%27s_Luck_Esp_v4.4_sample.pdf. [Consulta: 9 junio 2025].
- [7] Luis Llamas. *Tutoriales de Arduino* [en línea]. Dirección URL: <https://www.luisllamas.es/tutoriales-arduino/>. [Consulta: 3 julio 2025].
- [8] OpenJS Foundation. *Node-RED Documentation* [en línea]. Dirección URL: <https://nodered.org/docs/>. [Consulta: 6 junio 2025].
- [9] Programar Fácil. *Blog de Arduino* [en línea]. Dirección URL: <https://programarfácil.com/categoria/blog/>. [Consulta: 3 julio 2025].
- [10] Python Software Foundation. *Python Documentation* [en línea]. Dirección URL: <https://docs.python.org/3/>. [Consulta: 8 julio 2025].
- [11] W3Schools. *Tutoriales y referencias en línea* [en línea]. Dirección URL: <https://www.w3schools.com/>. [Consulta: 8 julio 2025].
- [12] Wokwi. *Simulador en línea de Arduino* [en línea]. Dirección URL: <https://wokwi.com/>. [Consulta: 10 julio 2025].